



郑州大学
ZHENGZHOU UNIVERSITY

PulseOS 初赛设计文档

郑州大学

石家誉 孔梦琪

2026 年 6 月

目录

第 1 章	概述	1
1.1	介绍	1
1.2	整体架构	1
第 2 章	内存管理	2
2.1	内存布局	2
2.1.1	内核虚拟地址空间	2
2.1.2	用户虚拟地址空间	2
2.1.3	多架构页表翻译与早期映射	3
2.2	内存分配	4
2.2.1	动态内存分配器选型与内存开销优化	5
2.3	虚拟内存机制	6
2.3.1	核心组件关系	6
2.3.2	映射后端类型	6
2.3.2.1	动态内存后端 Alloc	6
2.3.2.2	文件映射后端 File	7
2.3.2.3	写时复制后端 CowMapping	7
2.3.3	地址空间克隆与页表复制	8
2.4	内存锁定支持	9
第 3 章	进程管理	10
3.1	数据结构	10
3.2	任务调度	11
3.2.1	特殊任务	11
3.2.2	调度器策略与接口定义	11
3.2.3	定时器与时间片更新	12
3.2.4	异常处理与硬中断流程	13
3.2.5	系统调用处理与安全屏障	14
3.2.6	上下文切换与寄存器保存机制	15
3.3	生命周期管理	16
3.3.1	进程与线程的创建	16

3.3.2	vfork 挂起同步机制	16
3.3.3	进程映像替换 <code>execve</code>	16
3.3.4	进程退出与资源回收	17
第 4 章	文件系统	19
4.1	VFS 架构与描述符管理	19
4.1.1	FdTable 与 FdObject	19
4.1.1.1	多态实现	20
4.1.1.2	$O(1)$ 位图分配与指数扩容机制	21
4.1.2	共享描述符表并发访问控制	21
4.1.3	<code>exec</code> 时的 CLOEXEC 延迟释放	21
4.2	VFS 挂载树与路径越界解析机制	22
4.2.1	双向挂载边界越界解析机制	23
4.2.2	挂载传播与联动机制	24
4.3	ext4 驱动替换与文件系统性能优化	25
4.3.1	替换的背景	25
4.3.2	移植过程中 <code>axfs/vfs</code> 稳健性改进与优化	25
4.4	文件锁与 <code>flock</code> 机制	26
4.5	特殊文件系统	27
第 5 章	信号系统	29
5.1	信号队列及相关结构	29
5.1.1	核心数据结构	29
5.2	信号处理	30
5.2.1	信号排队与状态合并	30
5.2.2	信号分发	30
5.2.3	用户栈帧恢复	31
第 6 章	网络模块	33
6.1	套接字抽象与多态实现	33
6.2	Netlink 接口解析与构建	34
6.3	套接字控制 <code>Ioctl</code>	34
6.4	本地套接字退化与多态映射机制 <code>AF_UNIX</code>	35
第 7 章	阶段性成果与实现参考	37
7.1	阶段性成果	37

7.2	问题与经验	37
7.3	与同类系统亮点对比	37
7.3.1	物理页帧元数据管理	37
7.3.2	用户态内存拷贝安全隔离	38
7.3.3	纯 Rust 文件系统与异步/中断 I/O 架构	38
7.4	未来工作展望	38
7.5	实现参考	39
第 8 章	AI 使用说明	40
8.1	工具	40
8.2	模型	40
8.3	使用记录	40

第 1 章 概述

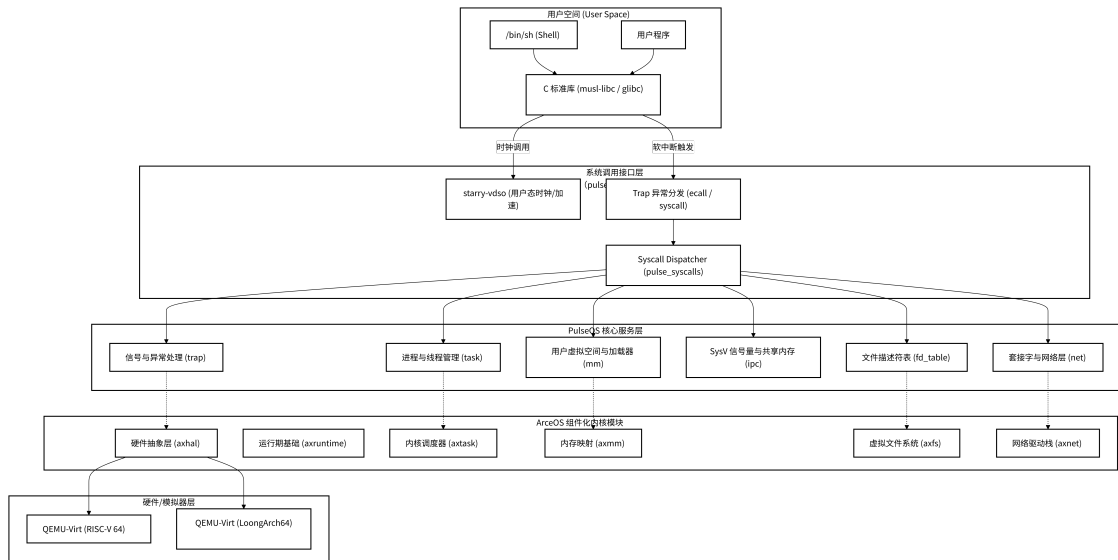
1.1 介绍

PulseOS 是一款使用 Rust 语言编写的、基于 ArceOS 组件化内核构建的、支持多架构的组件化宏内核操作系统。PulseOS 针对 RISC-V 64 与 LoongArch 64 双架构进行了深度适配与统一抽象，完整实现了进程管理、文件系统、内存管理、信号系统，以及网络模块。

PulseOS 提供了 POSIX 兼容的系统调用接口，以提供 Linux 兼容的功能，能够运行 musl-libc 与 glibc 编译的用户态应用程序，并且通过了 basic, busybox, cyclicttest, iotzone, iperf, libcbench, libctest, lmbench, ltp, lua, netperf 等初赛测例，并在部分性能测试项中取得了较为优秀的成绩。

1.2 整体架构

PulseOS 的整体软件栈可分为四个层次：



用户空间程序通过 musl-libc 或 glibc 提供的 C 库接口发起系统调用。当触发软中断时，系统陷入内核的 Trap 异常分发模块。系统调用派发器将控制权分发给内核服务层对应的模块，这些模块通过 ArceOS 的高度组件化基础单元提供硬件隔离的安全保障，并在 RISC-V 64 与 LoongArch 64 上完美运行。

第 2 章 内存管理

2.1 内存布局

PulseOS 区分了内核虚拟地址空间与用户虚拟地址空间，并在编译时根据生成的 `axconfig.toml` 对布局参数进行静态指定。对于物理地址原语，系统在 `memory_addr` 中的 `PhysAddr`、`VirtAddr` 和 `AddrRange` 等结构提供了强类型的静态编译保障，避免了裸指针带来的寻址混淆开销。

2.1.1 内核虚拟地址空间

内核虚拟地址空间配置参数如下：

参数	LoongArch64 配置值	RISC-V 64 配置值
物理内存基址 <code>phys-memory-base</code>	<code>0x8000_0000</code>	<code>0x8000_0000</code>
物理内存大小 <code>phys-memory-size</code>	<code>0x4000_0000</code>	<code>0x4000_0000</code>
内核镜像物理基址 <code>kernel-base-paddr</code>	<code>0x8000_0000</code>	<code>0x8020_0000</code>
内核镜像虚拟基址 <code>kernel-base-vaddr</code>	<code>0xffff_8000_8000_0000</code>	<code>0xffff_ffc0_8020_0000</code>
内核空间基址 <code>kernel-aspace-base</code>	<code>0xffff_8000_0000_0000</code>	<code>0xffff_ffc0_0000_0000</code>
内核空间大小 <code>kernel-aspace-size</code>	<code>0x0000_7fff_ffff_f000</code>	<code>0x0000_003f_ffff_f000</code>
物理-虚拟偏移量 <code>phys-virt-offset</code>	<code>0xffff_8000_0000_0000</code>	<code>0xffff_ffc0_0000_0000</code>

2.1.2 用户虚拟地址空间

用户地址空间的布局参数如下表所示：

区间名称	地址范围 / 基地址	说明与特性
------	------------	-------

内核虚拟地址空间	LA64: 0xffff_8000_0000_0000 RV64: 0xffff_ffc0_0000_0000 至 0xffff_ffff_ffff_ffff	1 GB 物理内存线性映射等
规范地址空洞	0x0000_003f_ffff_ffff 至: LA64: 0xffff_8000_0000_0000 RV64: 0xffff_ffc0_0000_0000	未使用的硬件规范虚拟地址空洞
保留/未映射区间	0x0000_0004_0000_0000 至 0x0000_003f_ffff_ffff	Sv39 理论用户态上限为 0x0000_003f_ffff_ffff，PulseOS 实际空间上限限制为 0x0000_0004_0000_0000
用户栈	0x0000_0004_0000_0000 向下增长	缺页自动向下拓展，拥有 1 MB 栈警戒区 <code>stack_guard_gap</code>
vDSO	0x7f00_0000 附近	ASLR 随机化基址，包含加速系统调用的用户态代码段
vvar	vDSO 紧下方，4 个页面 (0x4000 字节)	提供给用户态免陷入读取的时间、常数等内核映射数据
用户堆	0x0000_0000_4000_0000 向上增长	初始大小 64 KB，最大限制 512 MB
动态链接器/解释器	0x0000_0000_0400_0000	动态链接器加载基址
应用 ELF 代码段	0x0000_0000_0020_0000	采用位置无关可执行文件加载的应用代码/数据段
空指针保护区	0x0000_0000_0000_0000 至 0x0000_0000_0000_1000	用于捕获和防止空指针解引用的未映射保护页

2.1.3 多架构页表翻译与早期映射

PulseOS 支持在不同架构下实现统一的高级虚拟地址空间管理。其内核屏蔽了底层页表布局的具体差异，通过 `page_table_multiarch` 提供了强类型、与架构解耦的级联页表管理机制。

页表翻译与架构级特征：

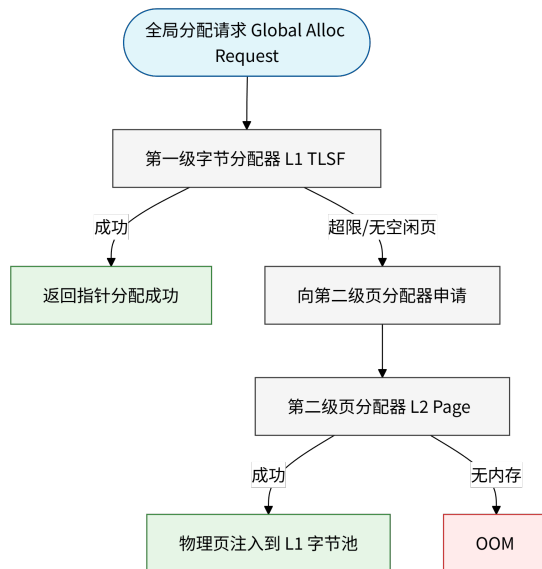
- **RISC-V 64**：采用 Sv39 模式。Sv39 提供了 39 位的虚拟地址空间，分为三级页表，以 4 KB 页为基本物理映射粒度。
- **LoongArch 64**：在用户态与内核态均采用多级页表寻址。而在引导期间，MMU 未启用时，LoongArch 64 依赖 DMW 来访问物理内存和外设。PulseOS 在引导期间配置了 CSR_DMW0（无缓存模式，用于外设 I/O）与 CSR_DMW1（缓存一致模式，用于物理内存）窗口，以支持在分页模式开启前的代码执行和栈环境建立。当 MMU 启用后，内核会切换至高端虚拟地址（0xffff_8000_...）运行，此后内核与用户态空间同样通过级联页表进行转换，以实现精细的页级页表防护与隔离。

引导初期的早期页表映射流程：

- **LoongArch 64 引导**：由于引导时 MMU 还处于半托管状态，内核引入了静态定义的早期页表，在 CPU 初次跳转至 C 语言运行期之前，手动将基本的串口等外设 I/O 寄存器地址与 RAM 基址进行静态恒等映射。
- **RISC-V 64 引导**：内核在 _start 段运行在物理地址，随后通过临时 Sv39 页表对内核映像进行双重映射，并将页表根物理地址写入 satp 寄存器。利用 sfence.vma 刷新 TLB 并通过指令跳转切换至高端地址空间运行。

2.2 内存分配

PulseOS 内存分配相关逻辑实现在 axalloc 模块中，采用了两级分配器架构和物理页帧引用计数表配合的分配机制。



2.2.1 动态内存分配器选型与内存开销优化

在系统开发初期，PulseOS 采用了默认的 Slab 与 Buddy 分配器。后续开发中为了优化内存占用与提升系统可靠性，我们进行了性能量化分析后，选择替换为 TLSF 分配器。

采集数据如下：

评估维度 / 测试项	Slab+Buddy	TLSF	内存与性能优化
内存常驻开销 <code>used_bytes</code>	11.24 MB	7.72 MB	-31.3%，内存节省
LoongArch64 Churn 吞吐	1242.50 K/s	1417.62 K/s	+14.1%，吞吐提升
LoongArch64 Realloc 吞吐	390.05 K/s	473.17 K/s	+21.3%，吞吐提升
RISC-V 64 Small Block 耗时	0.01228 s	0.01127 s	-8.2%，耗时缩减
lmbench: 进程 fork+exit 延迟	4.33 ms	3.28 ms	TLSF 领先 32.0%
lmbench: 进程 fork+execve 延迟	4.03 ms	3.50 ms	TLSF 领先 15.0%

两级分离检索的 Tuned TLSF 规避了 Slab 分配器的空闲链表检索，在原地扩容、进程创建、映像替换以及缺页异常处理中降低了延迟，加速了内核执行路径。

2.3 虚拟内存机制

2.3.1 核心组件关系

虚拟地址空间的组织结构由以下三个核心机制层层递进构成：

- **AddrSpace**：包含了虚拟地址范围，已注册的虚拟内存区域集合，以及锁保护的底层架构相关硬件页表实例。
- **MemorySet**：通用的虚拟内存区间管理容器。它维护虚拟内存区间起始地址到 **MemoryArea** 的 **BTreeMap** 映射，处理地址区间的映射、解映射（涉及区间的裁剪与拆分）以及区间权限保护属性的更新。
- **MemoryArea**：表示一段虚拟地址空间内连续且权限一致的区间范围，解耦了具体的物理页框分配，通过绑定映射后端控制访存行为。

2.3.2 映射后端类型

PulseOS 使用多种后端来控制虚拟内存到物理内存的映射动作：

后端类型	描述与实现机制
Linear	线性直接映射：在映射建立时一次性写入页表，不引发缺页异常。常用于内核关键数据段和外设 I/O 空间。
Alloc	动态内存分配：用于用户堆、栈及匿名映射。若不立刻分配物理内存，则采用懒加载通过缺页异常分配并清零物理页。
File	文件内存映射：通过 CachedFile 读写文件缓存。支持多进程共享页缓存和私有写时复制副本。
Cow	写时复制包裹：包裹任意只读/懒加载后端，用于在写入时触发写时复制机制。
Shared	共享内存：由 Arc<SharedFrame> 控制生命周期的物理连续页，多进程 shmat 映射时指向同一组物理地址。

2.3.2.1 动态内存后端 Alloc

Alloc 后端支持通过 **populate** 参数调节物理内存分配时机：

- 立即分配 (**populate = true**)：建立映射时立即填充物理页表，消除后续缺页开销。
- 延迟装载 (**populate = false**，默认)：初次访存时通过缺页异常按需装载。
 1. 若虚拟页完全未映射或为空占位符，调用 **alloc_frame(true)** 分配并清零。

2. 访问未映射页时，内核会主动探测并一并映射后续 3 个连续未映射页，减少内核态陷入开销，显著提升 `iozone` 顺序读写等测试的性能。

如果页面已被映射，但发生了权限升级请求（例如在只读页上发生了写访问），系统仅将当前拥有的物理页框附加新标志位，并调用 `pt_guard.remap` 执行原物理页的就地标志位升级，提高了非共享页修改权限的执行效率。

PulseOS 还实现了用户栈自动拓展。当用户栈向下增长时，对于带有 `grows_down` 属性的 `Alloc` 后端，内核会在缺页异常中自动为其向下分配和映射新物理页，并设置了 256 页的 `stack_guard_gap` 警戒区以防止越界覆盖。

2.3.2.2 文件映射后端 File

共享文件映射（`shared` 属性为 `true`）：

从 `CachedFile` 的全局页缓存中查询或分配物理页地址，然后将其直接映射到页表。由于底层物理页即为页缓存，对该虚拟内存的写入将直接反映在全局页缓存上。在 `unmap` 卸载时或通过 `msync` 等调用进行回写同步时，内核会将修改过的页面在 `CachedFile` 中标记为脏页，并调用 `sync()` 保证脏数据落盘。

私有文件映射（`shared` 属性为 `false`）：

初次读取/执行时，通过零拷贝机制直接映射全局 `Page Cache` 物理页，并清除 `WRITE` 属性，直至发生写访问时，再拷贝至通过 `alloc_frame` 新分配的进程私有物理页，以此保障私有修改不污染全局文件缓存。

2.3.2.3 写时复制后端 CowMapping

当且仅当发生的缺页异常带有 `WRITE` 属性且页表项中缺乏 `WRITE` 时触发该后端。

内核首先通过 `frame_table().get_ref(old_frame)` 获取该物理页当前的被共享次数，若该物理页不属于 `frame_table` 追踪范围，则默认共享数大于 1 以保守处理。

在引用计数为 1 时，说明已无其他进程共享该物理页，此时无须执行物理内存分配和数据复制，而是直接调用 `pt_guard.remap` 将当前页升级为可读写，从而提升独占 `COW` 页的写入效率。

若引用计数大于 1，则分配新的物理页框并将原页内容完整复制到新页中，更新页表指向新页框并开启写权限，随后调用 `dealloc_frame(old_frame)` 递减旧页

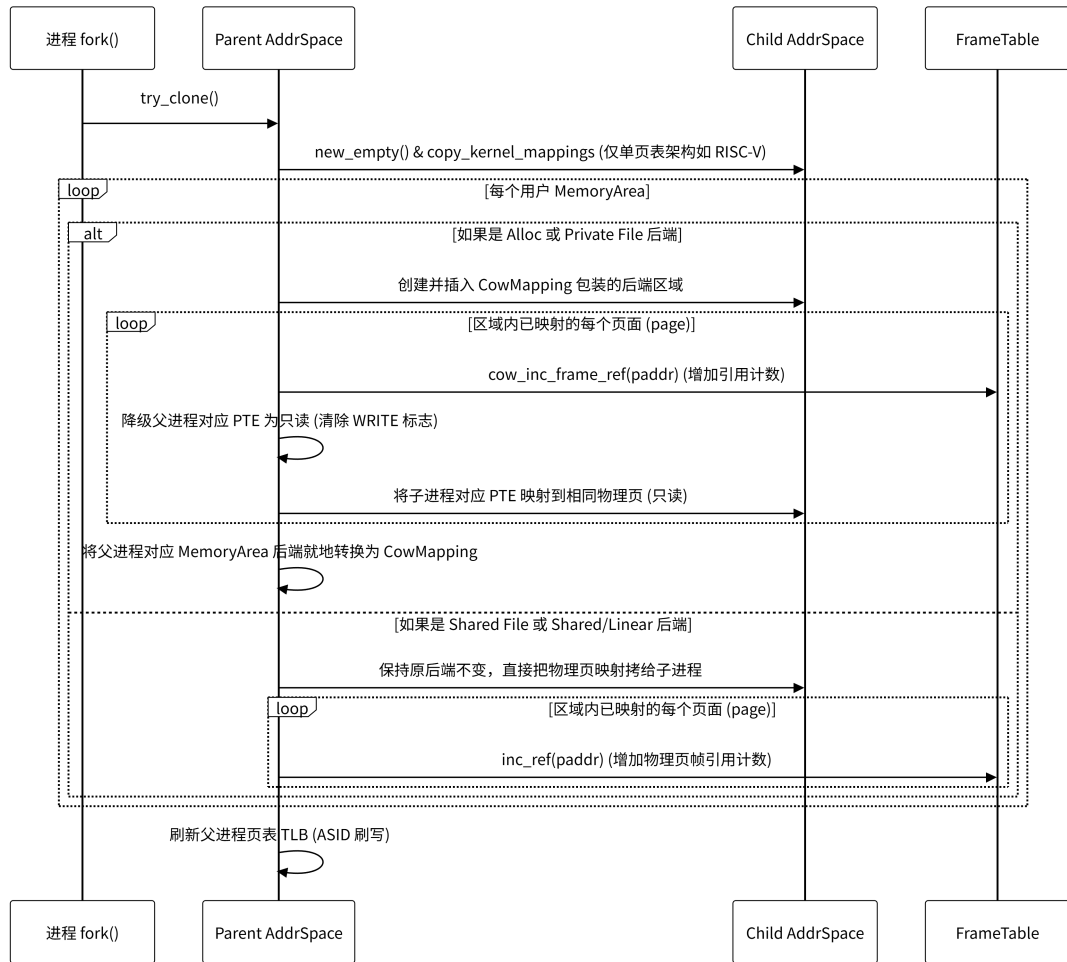
框的引用计数。若属首次访问，写时复制后端会委托内部包裹的文件或匿名分配后端执行初始分配映射。

2.3.3 地址空间克隆与页表复制

在 `fork` 调用中，`AddrSpace` 的克隆与页表复制并不是在高级层逐页进行的，而是通过修改的 `page_table_multiarch` 提供的递归克隆接口 `copy_cow_range` 高效实现：

- `copy_cow_range` 会自上而下递归遍历源页表，当目标页表未分配对应的中间级页表项时，会自动调用 `alloc_table` 分配并补全，动态复制出与源页表完全相同的层级路径，避免了高级层频繁陷入与映射的开销。
- 如果 `is_cow` 参数为 `true`，接口会检查源页表项。若源 PTE 可写，则清除写属性使其降级为只读，并在目标页表中以只读权限映射相同的物理页帧，使父子进程安全地共享同一块物理内存。
- 在映射物理帧时，通过传入的 `inc_ref` 回调闭包（即 `cow_inc_frame_ref`）增加物理页帧在全局 `FrameTable` 中的引用计数。当后续任何一方尝试写入该只读页面时，即可基于该引用计数触发 COW 缺页处理。

以 `fork` 为例，PulseOS 依赖 `try_clone` 克隆地址空间。在克隆过程中，各 `MemoryArea` 后端对底层的页表映射进行共享和克隆，构建 COW 关系：



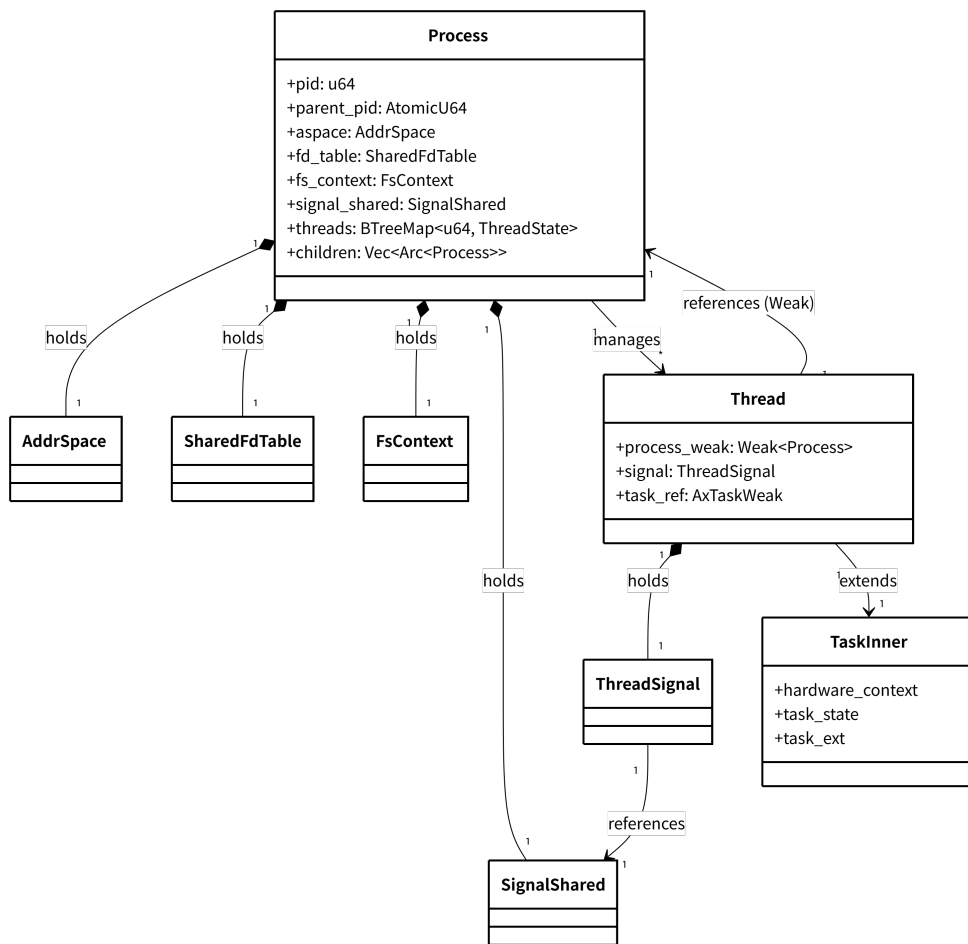
2.4 内存锁定支持

PulseOS 实现了 POSIX 兼容的内存锁定相关功能，如 `mlock`、`mlockall` 等系统调用。对于页面仅修改了虚拟元数据而未装载物理帧时会主动触发缺页处理，强制分配映射对应物理页，以保障实时任务的高优先级物理内存供给。

第 3 章 进程管理

3.1 数据结构

PulseOS 的进程/线程设计采用双层结构，实现了资源分配与调度执行的解耦。其核心实体包括 **Process**（进程控制块）、**Thread**（线程控制块）与 **TaskInner**（ArceOS 原生任务单元），三者的关联关系如下所示：



- **TaskInner** : ArceOS 内核感知的底层任务执行单元。负责保存任务运行时的硬件上下文及生命周期状态（**Running**、**Ready**、**Blocked**、**Exited**），并通过任务扩展机制绑定上层的 **Thread**。

- **Thread**：任务调度的基本单元。保存线程特有的信号掩码、调度策略、`clear_child_tid` 等状态信息。为了避免与 **TaskInner** 形成双向强引用环，**Thread** 内部通过弱引用指向 **TaskInner**，同时亦以弱引用关联其所属的 **Process**。
- **Process**：资源分配的最小单元。管理文件描述符表、用户虚拟地址空间以及该进程所属的所有线程集合等全局资源。

3.2 任务调度

3.2.1 特殊任务

PulseOS 在每个 CPU 核心初始化时会建立独立的运行队列并添加以下任务：

- **main** 任务：初始化时会向主 CPU 的运行队列中加入该任务，用于启动内核。
- **idle** 任务：当就绪队列为空时，PulseOS 就会进入该任务循环，直至有新任务进入就绪状态。
- **gc** 任务：对已退出的任务，调用 `exit_current` 时将其加入 **EXITED_TASKS** 队列并唤醒 **gc** 任务，**gc** 会立即释放内核栈空间，防止栈内存耗尽。

3.2.2 调度器策略与接口定义

PulseOS 中的调度策略接口被抽象为 **BaseScheduler Trait**，使用编译时 **feature** 指定调度策略，提供了以下接口：

- **add_task / remove_task**：添加或移除可运行任务。
- **pick_next_task**：选择并弹出下一个需要执行的任务。
- **put_prev_task**：将前一个运行的任务放回就绪队列，根据 **preempt** 参数决定是插在队头抢占位置还是队尾时间片耗尽与主动让出位置。
- **task_tick**：由定时器中断定期触发，推进当前任务的时间片并返回是否需要重新调度。

同时 PulseOS 在 ArceOS 提供的 **RRScheduler** 的基础上实现了优先级机制，并以此管理就绪任务。

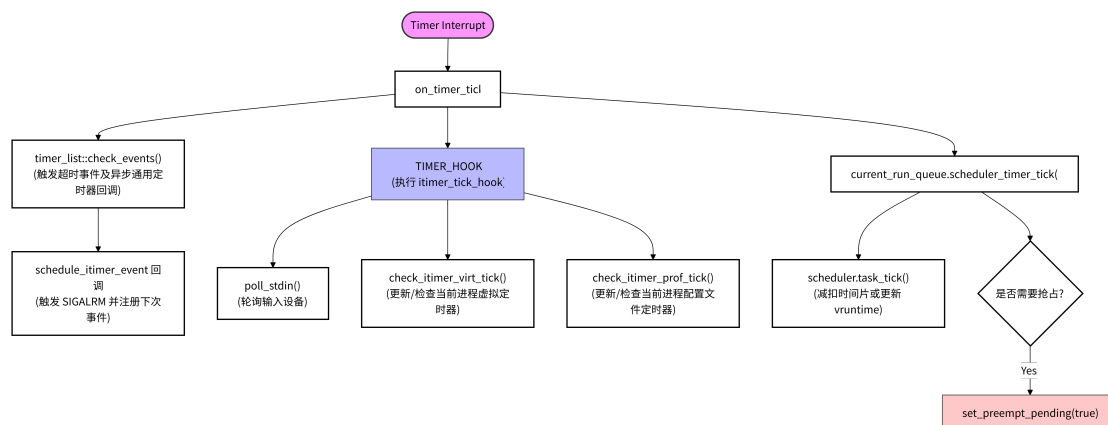
我们在就绪队列内部定义了 99 个实时 RT 优先级队列与 1 个普通优先级队列。任务通过 **priority** 和 **time_slice** 分别管理优先级和分配的时间片。

为了避免在高并发多任务下 $O(N)$ 遍历就绪队列，我们还在 **RRScheduler** 中引入了 **rt_bitmap: u128** 位图。每一位对应一个 RT 优先级队列，当某个队列中有任

务就绪时，位图中对应的位置为 1。调度器通过 `trailing_zeros()` 等二进制搜索函数，可以在 $O(1)$ 时间内快速定位当前具有就绪任务的最高优先级队列。

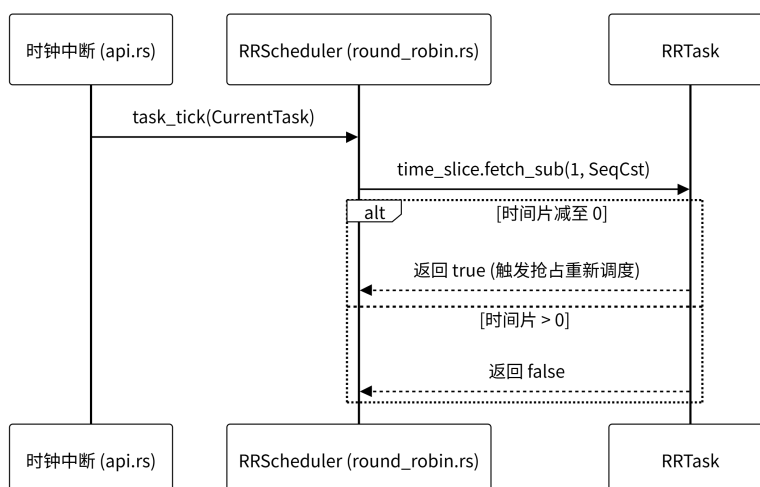
3.2.3 定时器与时间片更新

PulseOS 将定时器中断钩子注册到 `axtask` 中，其整体驱动流程如下：



在中断/异常处理完毕准备返回用户态的边界处，系统会自动检测当前任务的抢占挂起标志 `preempt_pending`。若该标志被置为 `true`（如当前任务时间片耗尽或被高优先级任务抢占），则触发 `preempt_resched()` 执行上下文切换，完成抢占调度流程。

而在上述流程的 `scheduler.task_tick()` 步骤中，对于采用时间片轮转的就绪任务，其时间片消耗与抢占判定的具体逻辑如下：



间隔定时器 `itimer` 机制

不同于基于时钟中断周期性 Tick 递减的虚拟/配置文件定时器（`itimer_virt`、`itimer_prof`），真实时间定时器 `itimer_real` 采用了异步的高精度管理与 `SIGALRM` 信号触发机制：

- 异步高精度触发：不同于基于 Tick 轮询递减的虚拟定时器 `itimer_virt` 与配置文件定时器 `itimer_prof`，`itimer_real` 采用高精度异步触发机制。用户设置定时器时，系统直接调用 `axtask::set_generic_timer` 向底层的时钟中断事件管理器注册事件回调，免去了在 Tick 钩子中进行全局扫描的开销。当时钟中断到达设定截止时间时，回调被精确触发并向进程发送 `SIGALRM` 信号。若设定了周期性时间（`interval > 0`），系统会自动计算下一次触发的 `deadline` 并递归重新注册，有效降低了每个 Tick 下的全局扫描与 CPU 遍历开销。
- 惰性唤醒失效机制（Timer Ticket ID）：为了防止已被非超时事件提前唤醒并激活的线程再次被后续的超时事件错误触发，PulseOS 在定时器注册时引入了唯一的 `ticket_id` 凭证。当线程因非超时事件提前唤醒时，其持有的 Ticket 会被标记为过期。当时钟中断回调触发时，系统会校验注册时的 Ticket ID 与当前线程状态的一致性，若检测到状态不一致，则直接丢弃该唤醒事件，保障了超时唤醒的准确性与安全性。

3.2.4 异常处理与硬中断流程

PulseOS 针对多架构，即 RISC-V 64 与 LoongArch 64，构建了高度一致的硬异常与中断处理架构。

硬件陷阱 Trap 发生与跳转：

- RISC-V 64：当硬件产生异常或中断时，硬件会将当前 CPU 特权级自动提升到 Supervisor 模式，将中断前的指令指针存入 `sepc` 寄存器，引发原因存入 `scause`，附加信息，如非法访存地址，存入 `stval`，并将特权状态映射到 `sstatus` 中。随后，CPU 跳转到由 `stvec` 寄存器指示的内核 Trap 向量入口 `trap_vector` 处。
- LoongArch 64：触发异常时，CPU 同样进入内核特权模式，将断点 PC 保存至 `CSR_ERA`，将异常类型写入 `CSR_ESTAT`，原特权状态写入 `CSR_PRMD`，并在包含映射错误时将非法地址填入 `CSR_BADV`。CPU 随后根据 `CSR_EENTRY` 指示的异常入口基址进行硬件跳转。

通用陷阱上下文 `TrapFrame` 保存与恢复：

在进入 `trap` 入口后，内核通过汇编代码首先保存当前任务的通用寄存器，即 RISC-V 的 `x1` 至 `x31`，LoongArch 的 `r1` 至 `r31`，到该线程的内核栈底部的 `TrapFrame` 结构中。为了支持多架构下的用户态切换与 `fork` 系统调用，在发生陷阱时必须正确恢复内核栈指针并对 `kernel_sp` 进行保护：

- RISC-V 64：在进入用户态前，将内核栈顶地址写入 `sscratch` 寄存器。陷入时，CPU 执行 `csrrw sp, sscratch, sp` 交换寄存器，从而恢复内核栈指针并把用户栈指针保存在 `sscratch` 中。
- LoongArch 64：利用自定义的 `KSAVE_KSP` 寄存器（映射到 `CSR_KSAVE0`，编号 `0x30`）来保护内核栈上下文。进入用户态前调用 `write_kernel_sp` 更新该寄存器；陷入时，汇编入口通过 `csrrd $sp, KSAVE_KSP` 恢复内核栈指针。

随后，内核将当前栈指针寄存器 `sp` 重定向为指向 `TrapFrame`，并调用 `trap_handler` 高级 Rust 分发入口。当分发执行完毕时，通过相反路径从栈底 `TrapFrame` 中恢复所有寄存器状态，最后执行物理返回指令，即 Supervisor 返回与 Exception 返回，重置特权级并返回断点。

3.2.5 系统调用处理与安全屏障

PulseOS 兼容了 Linux POSIX 系统调用规范，提供了 ABI 级别的二进制兼容。系统调用入口与 ABI 参数传递：

- 用户程序执行陷入指令，即 RISC-V 64 的 `ecall` 与 LoongArch 64 的 `syscall 0`，触发系统调用陷阱。
- 系统调用的 ABI 规范强制参数通过寄存器传递。参数 `syscall_id` 在 RISC-V 中由 `a7` 寄存器传入，在 LoongArch 中由 `a7` 传入；具体的系统调用参数在两架构中均统一由 `a0` 至 `a5` 寄存器传入。
- 系统调用返回后，错误码或返回值均回填到寄存器 `a0` 中。为了符合 Linux ABI 的兼容性设计，若执行失败，返回的错误码均取负数，如 `-EFAULT` 与 `-EINTR`，由 C 库，即 `musl-libc` 或 `glibc`，进行拦截并重填用户态的全局 `errno` 变量。
- Syscall 自动递增 PC：为了支持系统调用执行完毕返回用户态后能继续执行下一条指令，在处理系统调用异常（RISC-V 的 `UserEnvCall` 与 LoongArch 的 `Syscall` 异常）时，内核会首先将异常现场的程序计数器（`SEPC/ERA`）自动递增 4（即 `tf.sepc += 4` 与 `tf.era += 4`），然后再分发给具体的系统调用处理函数，以避免在返回用户态时重复触发陷入指令而陷入死循环。

为了实现物理的内核-用户态隔离，内核对用户传入的所有地址均执行严格的校验与受限拷贝：

- `validate_user_range` 边界验证：在读写任何用户内存，如路径字符串、读写缓冲区、信号上下文等之前，内核对传入指针区间进行严格的加法溢出检查，确保其起始地址与长度完全落在 `USER_SPACE_BASE` 至 `USER_SPACE_BASE + USER_SPACE_SIZE` 之间，任何非法的越界访问均被拦截并返回 `-EFAULT`。
- 数据拷贝安全保护：PulseOS 通过 `read_user_cstring_bytes` 与 `read_user_plain_array` 等接口提供数据拷贝屏障。在读取用户空间数据，例如文件路径与缓冲区内容，时，内核通过受保护的安全机制访问当前进程 `AddrSpace` 对应的页表与物理页面，以受控方式安全地拷贝数据，保护内核不受非法用户态地址指针的影响。

3.2.6 上下文切换与寄存器保存机制

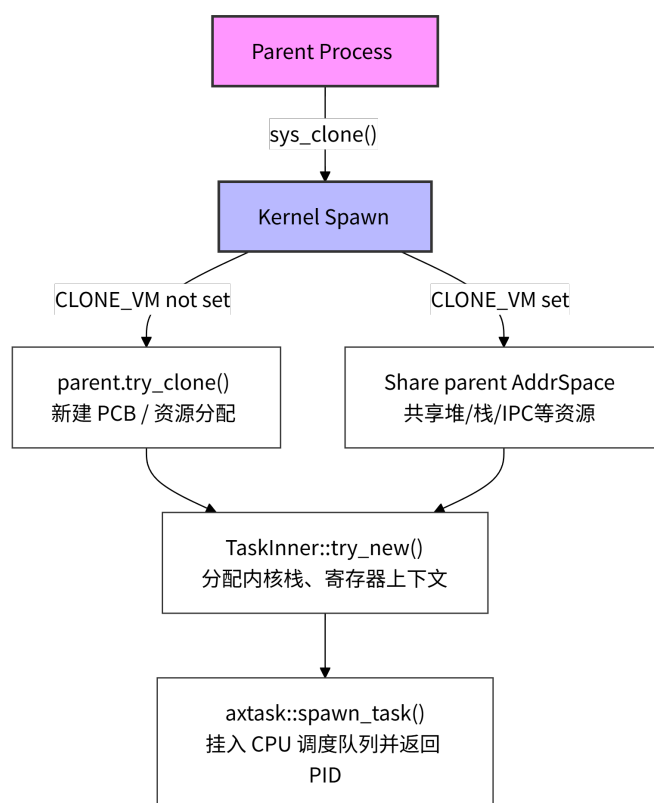
作为 PulseOS 的上下文切换机制由 ArceOS 提供，其细节设计如下：

- 运行队列与 `on_cpu` 跨核死锁预防：每个 CPU 核心拥有独立的运行队列 `AxRunQueue`。在多核 SMP 环境下，当 CPU A 调度切出任务 A 时，汇编代码将当前 CPU A 的硬件上下文保存到任务 A 的栈中。为了防止任务 A 在 CPU A 尚未完全完成切换时被 CPU B 瞬间唤醒并在 CPU B 的物理寄存器上运行，因为这会由于两核同时写入同一个内核栈导致严重的栈损坏，内核为每个任务引入了 `on_cpu` 原子状态标记。在 CPU A 彻底完成 `switch` 汇编跳转前，CPU B 如果尝试唤醒该任务，必须在 `clear_prev_task_on_cpu` 自旋等待该标记清空，从而杜绝了跨核并发栈损坏。
- 上下文保存汇编：底层利用汇编代码切换 CPU 的被调用者保存寄存器，其中 RISC-V 64 为 `s0` 至 `s11` 与返回指针 `ra`，LoongArch 64 为 `s0` 至 `s8`、`fp` 与返回指针 `ra`。
- 特权任务指针检索与 TLS：在 RISC-V 64 和 LoongArch 64 架构下，任务指针通过线程局部存储 TLS 的偏移量来进行硬件寻址。
- 惰性浮点寄存器切换：为避免频繁的浮点寄存器切换造成内存总线开销，系统在 RISC-V 64 架构下实现了惰性保存策略：只有当当前任务触发过浮点运算，导致特权寄存器 `sstatus.FS` 处于 `Dirty` 状态时，在切出时才保存浮点寄存器组；否则直接跳过，显著节省了纯整型计算线程的切换延迟。而 LoongArch 64 在配置 `fp-simd` 时，则采取了每次主动保存机制。

3.3 生命周期管理

3.3.1 进程与线程的创建

系统通过 `sys_clone` 与 `sys_clone3` 系统调用实现进程与线程的创建，支持线程与进程级别的资源克隆，具体创建流程如下：



3.3.2 vfork 挂起同步机制

当使用 `CLONE_VFORK` 创建子进程时，父进程与子进程共享相同的用户虚拟内存空间，且父进程必须被挂起，直到子进程释放该地址空间。

- 同步原语：父进程在克隆完子进程后，在 `vfork_context.event` 等待队列上阻塞。
- 解除挂起：子进程在成功替换页表后，或者在进程退出的时候，会主动调用 `complete_vfork()` 唤醒父进程。

3.3.3 进程映像替换 execve

进程映像替换由 `exec` 实现，其设计遵循半事务性的安全原则：

1. **预加载与验证:** 在释放当前地址空间之前, 先解析脚本解释器路径, 检测是否有其他进程正在写入该文件, 防止 `ETXTBSY`, 校验执行权限, 并对 ELF 头部格式进行初步校验。此阶段若出错, 可直接向用户态安全返回错误码, 而不破坏原进程。
2. **空间切换与原空间释放:** 分配一个全新的隔离地址空间 `new_aspace` 并映射初始的栈与堆。在此之后, 系统执行不可逆的切换操作——将进程的地址空间句柄替换为 `new_aspace`, 激活并切换当前 CPU 页表根节点, 随后立即释放原有的地址空间 `old_aspace` 以降低内存峰值。
3. **程序载入与单向终止:** 在激活新页表后, 才真正开始将新程序段及动态链接器载入内存。由于此时原地地址空间已被完全销毁释放, 若在载入阶段发生 ELF 损坏或其它加载期错误, 原进程状态无法恢复, 系统会在内核态直接终止当前进程, 而不会返回错误到用户态。
4. **清理旧状态:** 加载成功后, 断开所有原共享内存关联, 清空信号量 `undo` 队列, 关闭设置了 `FD_CLOEXEC` 标志的文件描述符, 并重置所有非忽略信号的行为为默认动作。

3.3.4 进程退出与资源回收

进程生命周期的终点是退出, 其拆卸和回收过程分为两个阶段:

1. 阶段一: 用户态资源同步回收

当进程的最后一个线程执行退出逻辑时, 由当前线程在内核态同步完成大部分用户空间资源的清理:

- **状态标记与同步唤醒:** 进程被标记为僵尸进程, 并调用 `complete_vfork()` 唤醒因 `CLONE_VFORK` 阻塞等待的父进程。
- **切换全局僵尸页表:** 为了避免当前运行的核心在彻底销毁进程页表时发生非法的内存地址访问, 系统使用全局唯一的空虚拟地址空间页表 `ZOMBIE_ASSPACE` 替换并激活当前物理页表, 随后立即释放并回收原进程的地址空间。
- **释放共享资源:** 同步卸载并断开所有共享内存关联, 清除信号量 `undo` 队列, 关闭进程中所有打开的文件描述符, 清理 `Futex` 等待队列, 释放所有内存锁定, 并重置文件系统上下文、精简用户凭证及进程执行参数。

2. 阶段二: 内核态资源异步回收

在线程完成第一阶段清理并触发 CPU 调度切换后, 进入内核资源的最终回收:

- **异步 GC 回收内核栈:** 已退出的线程实例被挂载到该 CPU 的全局 `EXITED_TASKS` 队列中。CPU 专属的异步垃圾回收协程 `gc_entry` 会被唤醒, 从队列中取出任务

并调用 `task.reclaim_kernel_stack()` 释放该任务占用的庞大内核栈，防止发生内存溢出。

- **僵尸进程收割与完全销毁：**父进程（或在父进程先死时托管的 `init` 进程）通过 `wait4 / waitpid` 系统调用，将子进程从其 `children` 列表中移除。当子进程 PCB 的所有强引用计数归零时，PCB 结构体及剩余的内核管理对象被彻底销毁。

第 4 章 文件系统

4.1 VFS 架构与描述符管理

4.1.1 FdTable 与 FdObject

PulseOS 中的虚拟文件系统采用面向对象的接口抽象设计，以支持多种文件类型的多态化处理。所有能够进行 I/O 操作的内核实体，如常规文件、目录、管道、网络套接字与事件描述符，都被统一抽象为描述符对象：

- **FdObject Trait:** 定义了对描述符的操作接口，包括 `read`、`write`、`ioctl`、`seek`、`poll` 以及可阻塞等待就绪的 `wait_ready`，为内核提供了极佳的解耦能力。FdObject 提供了一系列带有默认错误的实现，使得具体的实体对象仅需实现其所支持的操作。
- **FdTable 结构:** 包含一个存储 `Option<FdEntry>` 的动态向量数组 `entries`，以及一个基于位图的 `open_fds`。它利用位操作进行空闲描述符槽位的高效检索，并在空间不足时进行指数级动态扩容。

```
pub trait FdObject: Send + Sync {
    /// 用于向下转型为 dyn Any 以便获取具体实现类型
    fn as_any(&self) -> &dyn Any;

    /// 描述符的读取操作，默认返回 EBADF
    fn read(&self, _buf: &mut [u8]) -> LinuxResult<usize> {
        Err(LinuxError::EBADF)
    }

    /// 描述符的写入操作，默认返回 EBADF
    fn write(&self, _buf: &[u8]) -> LinuxResult<usize> {
        Err(LinuxError::EBADF)
    }

    /// 设备控制 I/O 操作，默认返回 ENOTTY
    fn ioctl(&self, _cmd: u32, _arg: usize) -> LinuxResult<isize> {
        Err(LinuxError::ENOTTY)
    }
}
```

```

}

/// 查询当前描述符的 I/O 事件就绪状态
fn poll(&self) -> LinuxResult<PollState>;

/// 移动描述符读写指针，默认返回 ESPIPE
fn seek(&self, _pos: SeekFrom) -> LinuxResult<u64> {
    Err(LinuxError::ESPIPE)
}

/// 可阻塞地等待描述符的指定事件就绪，默认返回 EOPNOTSUPP
fn wait_ready(&self, _events: i16, _deadline: Option<Duration>) ->
LinuxResult<bool> {
    Err(LinuxError::EOPNOTSUPP)
}

// ... 包含 stat, set_nonblocking, location 等其他辅助或特定类型的方法
}

```

代码 4-1 FdObject 抽象接口定义

4.1.1.1 多态实现

FdObject 支撑了 PulseOS 中丰富且多态的内核实体：

实体分类	具体实现结构	说明
标准 I/O 实体	StdinObject, StdoutObject	封装控制台 TTY 读写，内置 STDIN_WAIT_QUEUE 支持异步就绪唤醒。
文件系统实体	FileObject, DirObject	封装常规文件与目录的底层 axfs 系统调用交互。
进程与控制句柄	PidfdObject	进程文件描述符，提供跨进程信号发送与状态监测机制。
低延迟控制	CpuDmaLatencyObject	用于电源管理与低延迟 DMA 调优的专用描述符。目前底层驱动并未使用该限制

		值，其实现主要是为了满足 <code>cyclictest</code> 测例的运行要求。
IPC 与网络通信	PipeObject, Socket, EpollObject	分别对应管道双端、网络套接字以及事件多路复用轮询实体。

4.1.1.2 $O(1)$ 位图分配与指数扩容机制

为了高效管理海量描述符槽位，`FdTable` 采用二级管理机制：

1. **二级位图检索：**`open_fds` 存储为 `Vec<u64>`，每个 bit 对应一个描述符。在分配描述符时，通过对对应 `u64` 字的位操作定位空闲位。这能在单个字范围内实现 $O(1)$ 级时间复杂度检索。
2. **指数动态扩容：**系统初始化时 `entries` 保持较小规模。当申请的描述符超出当前向量容量时，系统将触发以 2 倍指数级自动对 `entries` 和 `open_fds` 进行扩容，最大扩增至内核硬上限 `FD_LIMIT` (1048576)，这既节省了冷启动内存，又保障了大并发场景下的平滑扩容。

4.1.2 共享描述符表并发访问控制

为了在多核多线程环境下保障描述符表的并发安全，PulseOS 采用了精细化的双重锁同步设计：

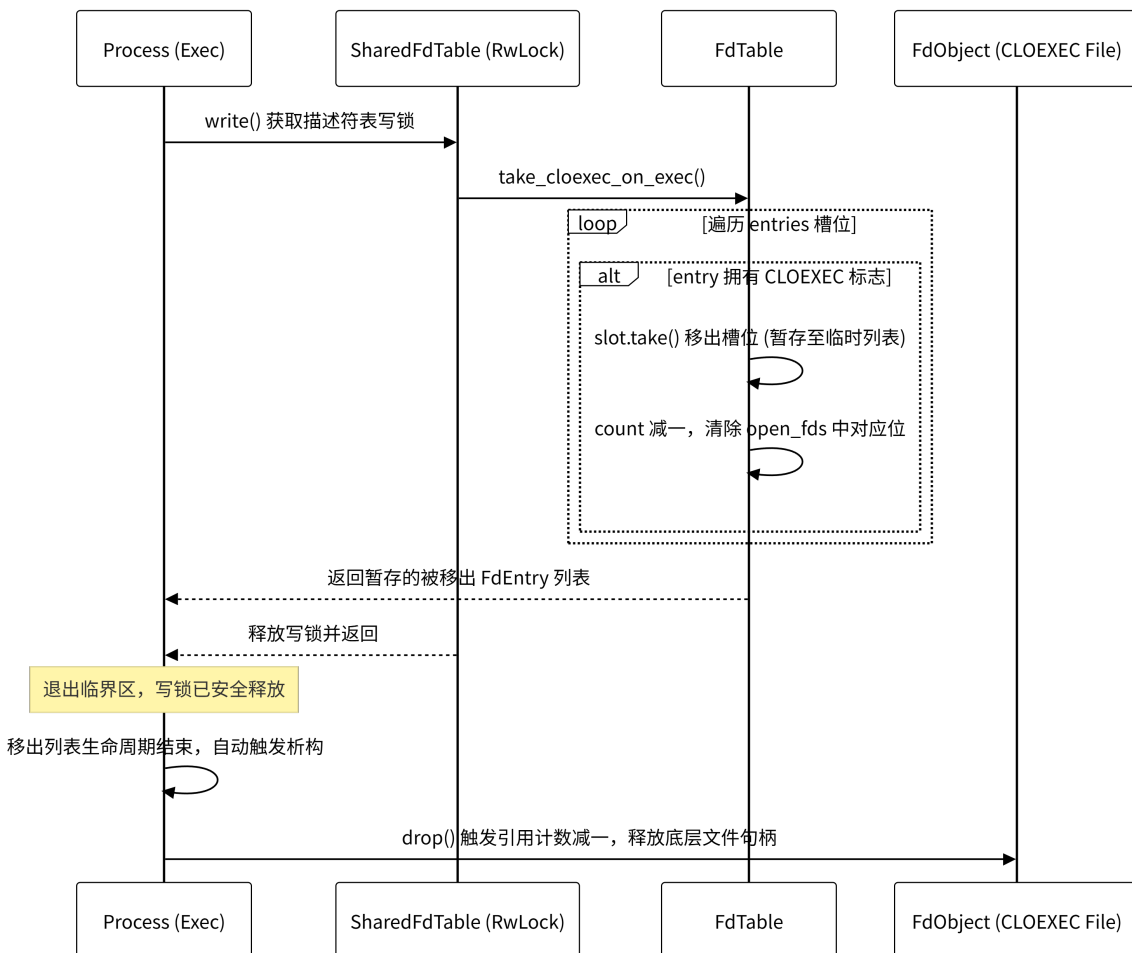
1. **表元数据同步锁**（`SharedFdTable = Arc<RwLock<FdTable>>`）：
 - 并发读锁：当线程执行常规 I/O 读写时，只需调用 `fd_table().read()` 获取读锁以取得 `Arc<dyn FdObject>`，允许并发操作不同的描述符，从而实现极高的吞吐率。
 - 排他写锁：当创建、复制或关闭描述符时，获取写锁 `write()`，以确保描述符表元数据（如 `entries` 的扩容、计数及位图）的一致性。
2. **进程级表指针锁**（`Process::fd_table: RwLock<SharedFdTable>`）：
 - 用于同步进程层面的描述符表切换。例如，当线程调用 `sys_unshare(CLONE_FILES)` 解除与父进程的描述符共享时，通过获取该锁的写锁来安全地将进程的 `SharedFdTable` 指针替换为私有拷贝，防止表替换与多线程并发访问发生竞态。

4.1.3 `exec` 时的 `CLOEXEC` 延迟释放

在执行 `exec` 时，系统会调用 `take_cloexec_on_exec()`，查找所有带有 `CLOEXEC` 标志的描述符项并将其从表中移除。为了缩短写锁的持有时间，PulseOS 在设计上采用了延迟析构策略：

关闭文件描述符或网络套接字等实体可能会伴随着耗时的 I/O 同步、缓冲区刷新或底层设备通信。如果直接在持有一级写锁的临界区内释放这些实体，将造成严重的锁阻塞。因此，我们选择使用 `take_cloexec_on_exec` 在临界区内将所有带有 `CLOEXEC` 标志的描述符项提取并打包到一个临时 `Vec<FdEntry>` 向量中并返回。当离开作用域释放写锁后，在临界区外对该向量执行 `drop`，完成底层 `FdObject` 引用计数的归零及物理句柄的真正释放。

这一流程的时序关系如下：



4.2 VFS 挂载树与路径越界解析机制

PulseOS 支持高度灵活的多源文件系统挂载与传播机制,这一部分主要由 AI 在 LTP 中 `fs_bind` 及相关测例中实现。

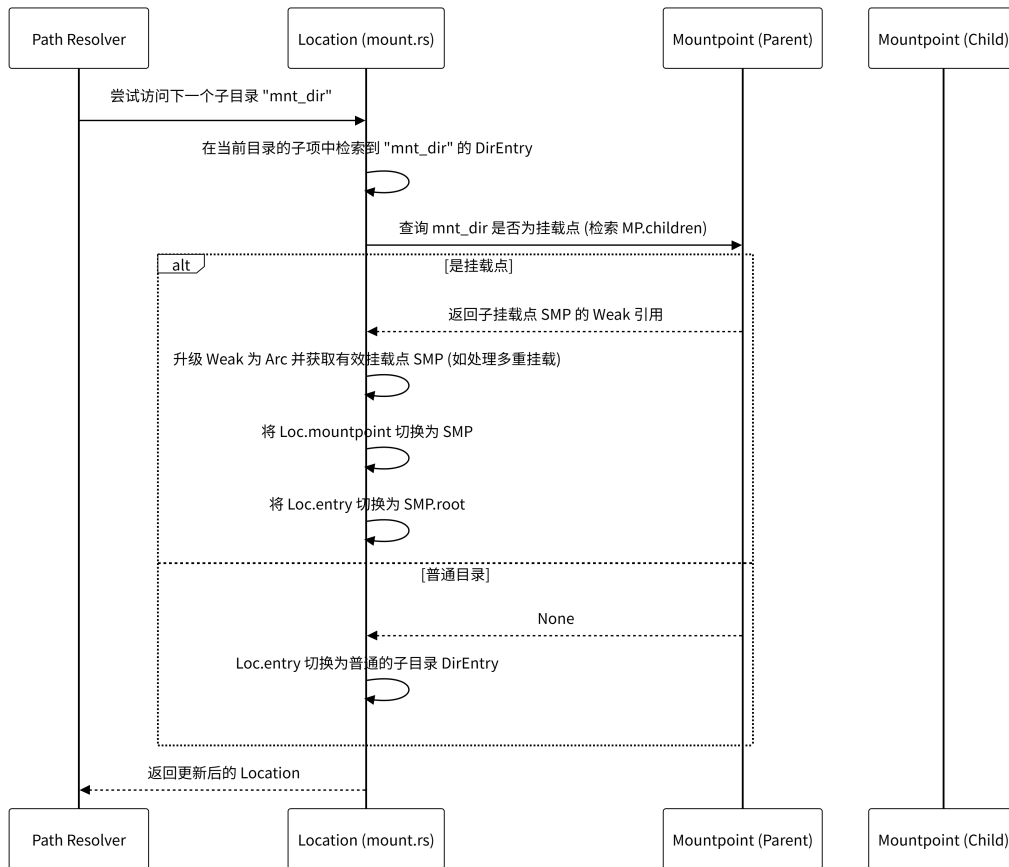
- **挂载节点 Mountpoint**：代表文件系统的挂载实例。它并没有直接存储目标文件系统引用，而是拥有挂载点的根目录项 `root: DirEntry`（其内部已绑定特定文件系统）。它不直接拥有父挂载点引用，而是通过包装了父挂载点上下文的路径游标 `location: Option<Location>` 间接回溯；同时，它通过子挂载点弱引用集合 `children: Mutex<HashMap<(u64, usize), Weak<Mountpoint>>>` 来建立挂载树树枝，以 `(inode, fs_ptr)` 对作为唯一键。使用 `Weak` 弱引用可避免父子挂载点循环引用导致的内存泄漏。
- **路径游标 Location**：包含了当前的节点 `entry: DirEntry` 以及它所属的挂载实例 `mountpoint: Arc<Mountpoint>`。当路径解析跨越挂载边界时，`Location` 会自动切换 `Mountpoint` 上下文。
- **挂载传播**：PulseOS VFS 实现了完整的挂载传播状态机，支持共享挂载（`Shared`）、从属挂载（`Slave`）、私有挂载（`Private`）与不可绑定挂载（`Unbindable`）等类型，并能在挂载/卸载时在对等组（`Peer Group`）成员间进行递归传播或联动清理。

4.2.1 双向挂载边界越界解析机制

路径解析在跨越挂载点边界时表现为双向解析：

1. **向下越界（进入挂载点）**：当路径解析器尝试访问下一个子目录（如 `mnt_dir`）时，系统会先在当前目录的子项中检索出相应的 `DirEntry`。接着，在当前挂载点的 `children` 映射表中以该 `DirEntry` 的 `entry_key` 即 `(inode, fs_ptr)` 作为键进行查找。如果命中（即该子目录为挂载点），则将 `Location` 切换为子挂载点实例，并将 `entry` 指向子文件系统的根目录。

时序关系如下所示：



2. **向上越界（回溯至父挂载点）**：当在当前挂载点的根目录下解析父目录时，若当前 `Location` 已是当前挂载点的根目录（即 `is_root_of_mount()` 为真），路径解析器将回溯其绑定的挂载位置 `mountpoint.location()`，安全跨越边界返回到父挂载点中挂载该子挂载点的那个父目录的 `Location`，从而实现挂载树的无缝逆向遍历。

4.2.2 挂载传播与联动机制

PulseOS 支持复杂的挂载传播机制，通过一个全局的对等组注册表 `PEER_GROUPS: Mutex<HashMap<u64, Vec<Weak<Mountpoint>>>>` 维护共享组关系：

- **传播类型**：包含 `Private`（无传播）、`Shared`（与对等组成员同步共享）、`Slave`（接收来自 `Master` 的传播，但不反向传播）、`SharedAndSlave`（混合模式）以及 `Unbindable`（防止无限绑定复制）。

- **挂载子树递归传播**: 在对对等组进行挂载时, 若目标父挂载点处于共享模式, 新挂载会通过 `propagate_new_mount` 生成副本并递归复制挂载子树, 传播至组内所有成员。
- **联动卸载**: 在执行 `unmount` 时, 系统会递归收集并联动卸载所有因挂载传播而在其他对等组成员中自动生成的关联挂载, 保证系统状态的一致与清洁。

4.3 ext4 驱动替换与文件系统性能优化

4.3.1 替换的背景

为了实现更加健壮和高效的持久化存储支持, 我们在开发后期决定将 `ext4_rs` 替换为 `ext4plus`, 原因有以下几点:

- `ext4_rs` 驱动在设计上是完全同步的, 这导致其极难适应高并发或未来可能引入的异步 `async/await` 磁盘 I/O 运行时。而 `ext4plus` 在库级层面实现了高度的同步/异步编译解耦, 能无缝适配异步块层, 为 PulseOS 后续向全异步与中断化文件系统的演进奠定了基础。
- `ext4_rs` 默认采用直写机制。我们曾为其引入了缓存层, 但暴露出数据不一致以及死锁竞态等安全隐患。此外, 它也缺失了许多 `ext4` 的高级特性; 尽管我们实现了动态块大小等特性, 但在进一步尝试支持日志机制时遇到了大量问题, 这促使我们决定将其彻底替换。
- 在方案的选择上, 我们曾尝试过 ArceOS 默认的 `lwext4_rust` 并进行了性能对比。虽然其读写速度与当时的 `ext4_rs` 基本持平 (当时的性能瓶颈实际上主要在 VFS 层), 但考虑到其缺乏 `ext4plus` 优秀的异步扩展性, 我们最终放弃了该方案。

4.3.2 移植过程中 axfs/vfs 稳健性改进与优化

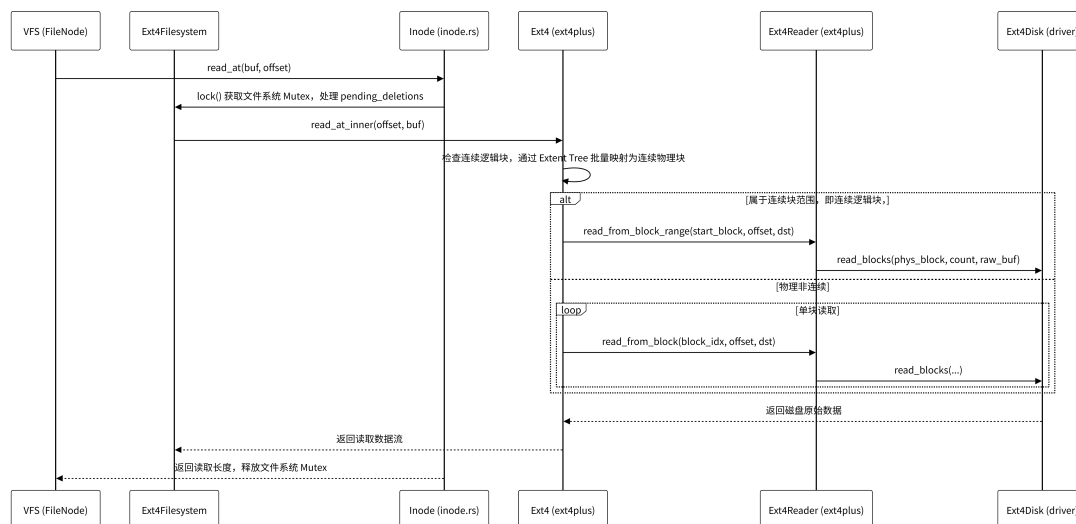
在将 `ext4plus` 整合进 PulseOS 虚拟文件系统的过程中, 我们对 `axfs` 和 `vfs` 层进行了深度的健壮性改进与性能优化。

在安全方面, 我们将磁盘 I/O 锁由自旋锁升级为睡眠锁, 允许 I/O 等待时出让 CPU 以防范忙等死锁; 同时, 引入了 Inode 缓存命中时的父级弱引用更新机制、防重入的全局刷盘任务过滤器 `FLUSHING_TASKS`, 以及用于推迟已删除文件物理释放的 `pending_deletions` 延迟回收队列, 有效解决了锁重入死锁、内存泄漏和解挂载漏洞。

在性能提升方面，我们设计了 8 槽位最近关闭文件缓存，实现了 4KB 对齐的 DMA 零拷贝直通读写；同时，通过优化 `query_user_page_slice` 大缓冲区边界的一次性验证，以及 `Futex` 调度锁前的物理页预翻译，大幅消除了 VMA 检索开销，并规避了调度临界区内的缺页死锁风险。

在这一系列改进实施后，我们使用 `qperf` 对 `iozone` 中的各项测试进行了性能数据采样。`iozone` 测试中 57.22% 的 CPU Ticks 处于空闲等待状态，这主要源于内核主任务在等待 I/O 完成时的阻塞。这表明当前的性能瓶颈已转移到慢速磁盘 I/O 硬件本身导致的任务阻塞上。因此，PulseOS 的下一阶段改进计划将从异步化与中断化着手。

从 `ext4plus` 文件中读取数据块的链路如下：



4.4 文件锁与 flock 机制

PulseOS 实现了标准的 `flock` 文件锁语义，用于在多个进程或线程之间同步对同一个文件的访问：

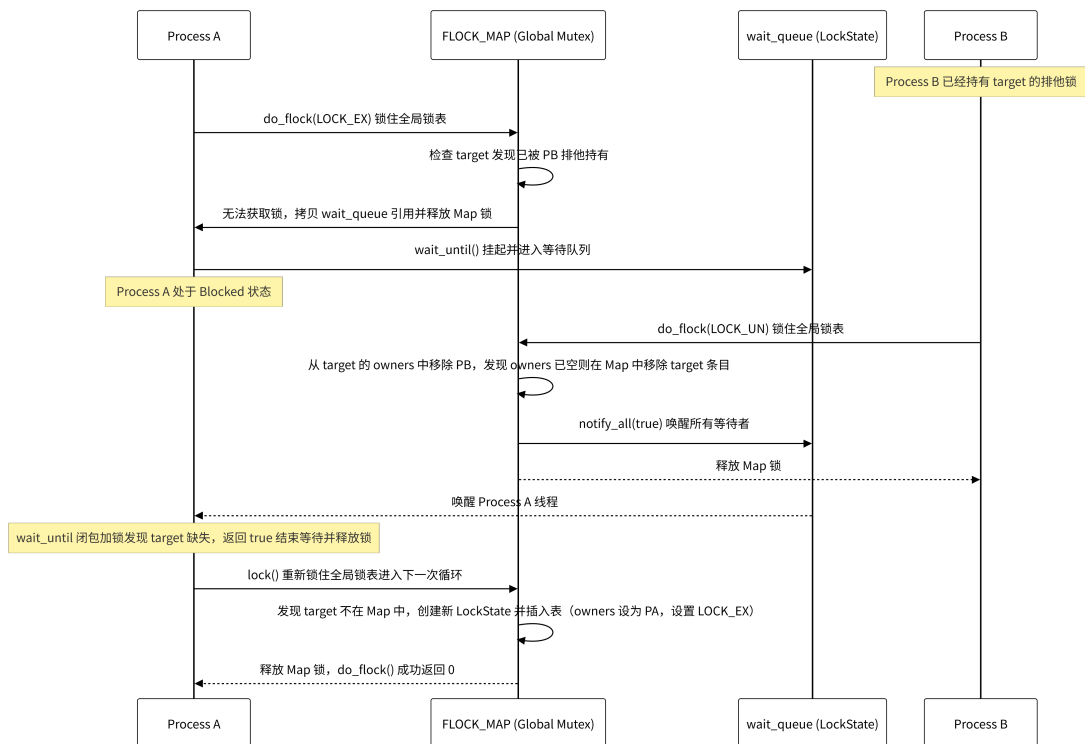
锁目标识别 `LockTarget`：支持两种锁定颗粒度：

- **Location**：基于底层文件系统的 `fs_id` 和文件的 `inode` 号进行识别。这是标准的文件锁，所有指向同一物理文件的描述符共享同一个锁状态实例进行争用；但在由 `dup / fork` 复制的描述符之间由于共享同一个描述符对象，因而共享该锁的持有权。
- **FdObject**：基于描述符对象的内存指针地址进行识别。

锁状态 `LockState` 包含锁的类型，即 `Shared` 共享锁或 `Exclusive` 排他锁，当前持有此锁的所有者列表 `owners`，以及用于阻塞等待的等待队列 `wait_queue`。全局锁表 `FLOCK_MAP`：所有的文件锁都被汇总存放在一个全局静态的 B 树映射表 `FLOCK_MAP: Lazy<Mutex<BTreeMap<LockTarget, LockState>>>` 中，由一个 `spin::Mutex` 锁进行全局保护。

在阻塞等待锁释放的过程中，`do_flock()` 会在每次被唤醒时通过 `current_has_pending_signal` 检查当前线程是否有挂起的未屏蔽信号。若有，则立即返回 `EINTR` 中断错误，支持标准的 POSIX 信号打断语义。

进程 A 申请排他锁被阻塞，等待进程 B 释放锁后被唤醒的流程如下：



4.5 特殊文件系统

PulseOS 在 `axfs/src/fs` 中实现了多种特殊文件系统，用来向用户态提供运行期状态以及虚拟硬件接口：

1. `procfs` 进程文件系统：

- `/proc/self`：动态指向当前发出请求的进程 PID 目录的符号链接。

- `/proc/meminfo` : 向用户态 `free` 等工具动态回填系统内存总量、空闲量及可用量。
 - `/proc/filesystems` 和 `/proc/cpuinfo` : 动态回填当前系统支持的文件系统列表及 CPU 处理器硬件信息。
 - `/proc/sys/kernel/` : 提供 `pid_max`、`tainted`、`core_pattern` 等内核配置状态节点。
 - `/proc/[pid]/maps` : 通过遍历进程 `MemorySet` 中的各个虚拟区域, 动态拼接虚拟地址段、权限、偏移量、设备号、Inode 以及关联文件路径, 格式化输出给用户态程序。
 - 进程描述符扩展: 在 `/proc/[pid]/` 目录下支持 `fd/`、`ns/(stub)`、`children` 以及 `task/`。
2. `devfs` 设备文件系统:
- 动态提供 `/dev/null`, 该节点会丢弃所有写入并在读取时返回 EOF;
 - `/dev/zero`, 能无限提供 0x00 字节; `/dev/tty` 与 `/dev/console`, 提供控制台字符通道输入输出。
 - 提供 `/dev/random` 与 `/dev/urandom`, 通过 PCG 随机数生成算法提供伪随机数流。
 - 设备控制及虚拟硬件: 提供 `/dev/cpu_dma_latency` 和 `/dev/loop-control`, 并能将系统检测到的磁盘等块设备动态注册为 `/dev/vd*` 块设备节点。
3. `tmpfs` 临时文件系统:
- 支持高并发的临时文件创建与读写。由于采用全内存读写, 且与 VFS 层的页缓存系统深度集成, 在检测到 `tmpfs` 时会自动实例化无容量限制的内存页缓存作为介质, 保证了编译、解压等测试项的高吞吐性能。

第 5 章 信号系统

5.1 信号队列及相关结构

PulseOS 的信号子系统对接了标准 POSIX 信号的核心机制。为了在多线程环境下实现高效、合规的信号路由与过滤，我们将信号状态划分为进程共享与线程局部两级，并使用多级原子挂起位图进行管理。

5.1.1 核心数据结构

1. **SignalShared**：管理进程内所有线程共享的信号处理程序、进程级未决信号以及相应的信号附加信息。其定义如下：

```
// pulse_core/src/task/signal.rs
pub struct SignalShared {
    handlers: Arc<SignalHandlers>,
    process_pending: AtomicU64,
    pub pending_sinfo: Mutex<BTreeMap<usize, [u8; 128]>>,
}
```

代码 5-1 SignalShared 进程共享信号状态结构定义

2. **ThreadSignal**：每个线程独占的信号状态，包括线程私有的未决信号、信号屏蔽字、是否处于 Handler 执行中的状态标志、嵌套恢复上下文以及替代信号栈（altstack 目前只实现了接口与注册，并没有实现真正的替换逻辑）。其定义如下：

```
// pulse_core/src/task/signal.rs
pub struct ThreadSignal {
    shared: Arc<SignalShared>,
    thread_pending: AtomicU64,
    blocked: AtomicU64,
    in_handler: AtomicBool,
    skip_once: AtomicBool,
    signal_wait: WaitQueue,
    saved_ctx: Mutex<Option<SavedSignalContext>>,
    altstack: Mutex<SignalAltStack>,
```

```

    sigsuspend_restore: Mutex<Option<u64>>,
    pub pending_siginfo: Mutex<BTreeMap<usize, [u8; 128]>>,
}

```

代码 5-2 ThreadSignal 线程局部信号状态结构定义

5.2 信号处理

5.2.1 信号排队与状态合并

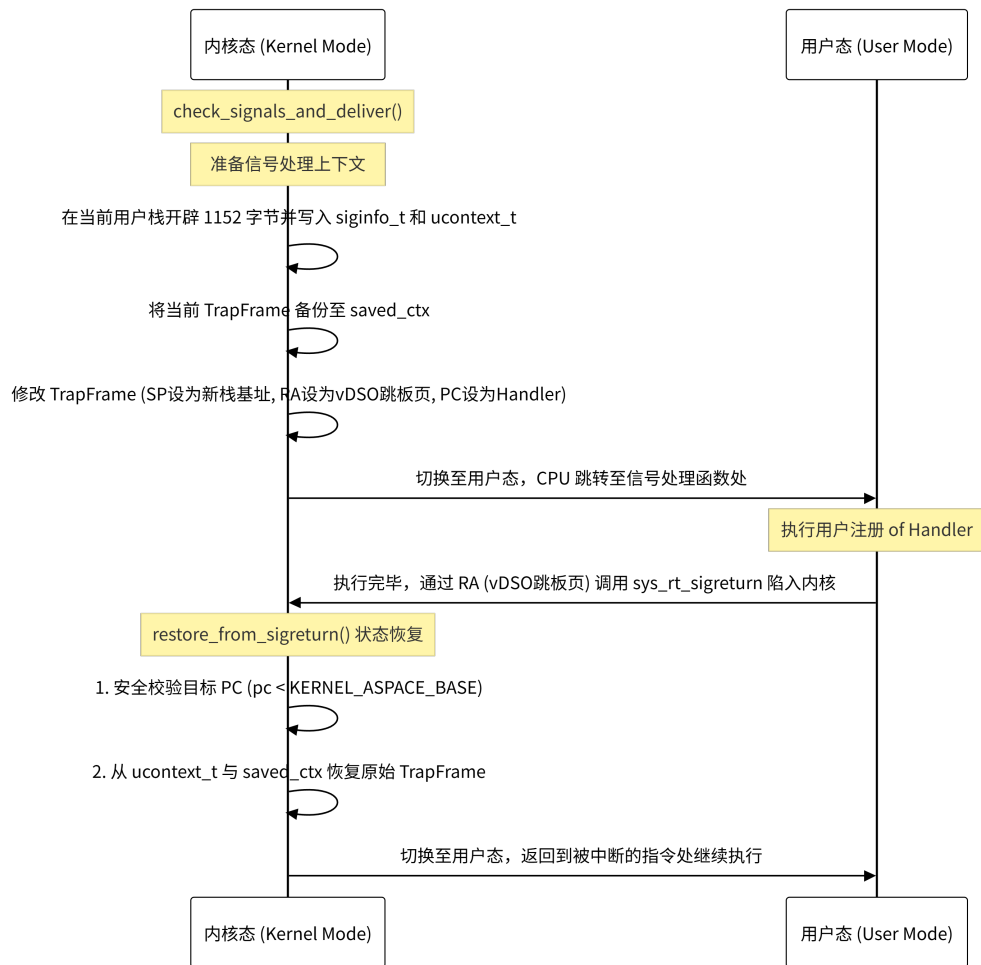
当用户执行 `kill`、`tkill` 或 `tgkill` 发送信号时，内核会计算出目标线程，并更新相应的 `pending` 位图，随后唤醒其所在的阻塞队列。

在每次线程即将从内核态返回用户态前，内核会将线程局部的 `thread_pending` 与进程级的 `process_pending` 进行按位或运算，并过滤掉被屏蔽的信号，从而合并计算出当前真正可递送的信号集合。

5.2.2 信号分发

对于上述可递送集合中自定义了处理程序的信号，内核将在返回用户态前，通过以下步骤注入信号入口并启动处理流程：

- 栈帧开辟：内核在当前用户栈上，向下开辟 `frame_size = 1152` 字节空间并按 16 字节对齐，将当前 `TrapFrame` 的通用寄存器和信号屏蔽字打包写入用户栈的 `ucontext_t` 与 `siginfo_t` 结构中。
 - 修改上下文：
 - 将 `PC sepc`（RISC-V 64）或 `era`（LoongArch 64）重定向为用户注册的函数指针 `act.handler`。
 - 将通用寄存器 `a0` 设为信号编号 `sig`，`a1` 设为 `siginfo_t` 用户栈地址，`a2` 设为 `ucontext_t` 用户栈地址。
 - 将栈指针 `sp` 设为刚刚开辟的用户栈帧基址。
 - 蹦床注入：将返回地址 `ra` 改写为进程注册的 `signal_trampoline` 地址，即 `vDSO` 中的 `__vdso_rt_sigreturn` 的地址。
 - 信号处理开始：退出内核进入用户态，CPU 会直接跳转到信号处理函数处执行。
- 信号处理的整体生命周期时序与控制路径如下时序图所示：



5.2.3 用户栈帧恢复

当用户信号处理程序执行完毕时，通过 `ra` 跳转进入 vDSO 的 `__vdso_rt_sigreturn` 系统调用重新陷入内核。内核在系统调用处理函数中恢复通用寄存器和执行状态的流程如下：

- 上下文恢复流程：
 - 内核首先从内核安全存储区 `saved_ctx` 中取出被信号打断时的原始寄存器状态，并完全写回当前的 `TrapFrame`。
 - 随后，内核尝试从用户栈上的 `ucontext_t` 结构中读取用户可能修改的通用寄存器，并安全地覆盖更新至 `TrapFrame` 中。
- PC 校验与防御：在将用户态修改的指令指针应用回 `TrapFrame` 前，内核强制执行 `PC` 地址的边界防御校验：

- 读回的 PC 必须指向用户地址空间。若检测到读回的 PC 指向内核地址空间，内核将拦截该跳转，并安全回退至 `saved_ctx` 中原本保存的被打断指令地址继续执行。这作为纵深防御的一部分，确保了即使用户态的 `ucontext_t` 被恶意篡改，控制流也无法被劫持到内核地址空间，保障了内核的安全边界。

第 6 章 网络模块

PulseOS 的网络模块实现大部分工作由 AI 完成。

6.1 套接字抽象与多态实现

网络模块在 `pulse_core/src/net/mod.rs` 中被统一抽象并集成至 VFS 多态接口下。其核心的 `Socket` 结构体包含具体的 `SocketInner` 枚举，统一了对不同网络协议类型的控制：

```
// pulse_core/src/net/mod.rs

pub struct Socket {
    pub domain: AtomicU32,
    pub inner: SocketInner,
    pub pending_send: Mutex<alloc::vec::Vec<u8>>,
    pub pending_addr: Mutex<Option<core::net::SocketAddr>>,
    pub rx_shutdown: AtomicBool,
    pub tx_shutdown: AtomicBool,
}

pub enum SocketInner {
    Tcp(TcpSocket),
    Udp(UdpSocket),
    Local(LocalSocket),
    Packet(PacketSocket),
    Netlink(NetlinkSocket),
}
```

1. Tcp / Udp 套接字：

- 封装底层的 `axnet` 驱动，并调用 `smoltcp` 完成 IP、TCP、UDP 协议栈的数据包分发、解析和滑动窗口维护。

2. Packet 原始套接字（Mock 桩实现）：

- 提供 `AF_PACKET` 类型的原始链路访问抽象，支持相关的套接字配置选项。目前为 Mock 占位实现，写操作会直接丢弃数据包并报告成功，读操作返回 `0`。

3. Netlink 通道（静态 Mock 实现）：

- 提供了对 AF_NETLINK 协议族的支持。目前采用静态响应机制，主要用于解析用户空间的网络配置查询请求，硬编码返回静态的本地环回接口 lo 和以太网网卡 eth0（IP 默认为 10.0.2.15，MAC 默认为 52:54:00:12:34:56），以保证用户空间的网络管理与查询工具能顺利读取配置。

6.2 Netlink 接口解析与构建

为了向用户态网络管理程序提供网络状态报告，NetlinkSocket 实现了对 RTM_GETLINK 和 RTM_GETADDR 消息的静态 Mock 支持：

- 当用户态向 Netlink 发送写请求时，内核解析 Netlink 消息头部的 16 字节，提取消息类型与序列号。
- 静态 Mock 状态生成：
 - RTM_GETLINK：硬编码构建 ifinfomsg 结构及嵌套的 RTA 属性，静态返回本地环回接口 lo 和以太网网卡 eth0 的链路状态。
 - RTM_GETADDR：硬编码构建 ifaddrmsg 结构，静态返回 lo 接口的 127.0.0.1 地址和 eth0 网卡的 10.0.2.15 地址。
- Netlink 结构定义：

```
pub struct NetlinkSocket {
    pub rx_data: Mutex<alloc::vec::Vec<u8>>,
    pub read_offset: Mutex<usize>,
    pub nonblocking: AtomicBool,
}
```

- 将组装好的静态 Netlink 消息存入 rx_data 缓冲区，供用户态随后通过读系统调用读取，从而使 iproute2 工具链中的基础查询指令能在 PulseOS 上顺利运行并获取模拟配置。

6.3 套接字控制 Ioctl

在套接字的 ioctl 系统调用处理中，PulseOS 提供了对网络测试及工具链关键控制命令的静态 Mock 兼容：

- FIONREAD：用于读取当前网络套接字接收缓冲区中未读的数据字节数。内核通过调用 recv_queue() 得到缓冲长度（如 TCP/UDP 底层队列，或 UNIX 域套接字的环形缓冲区可读大小），并将长度写入用户态指定的参数指针。

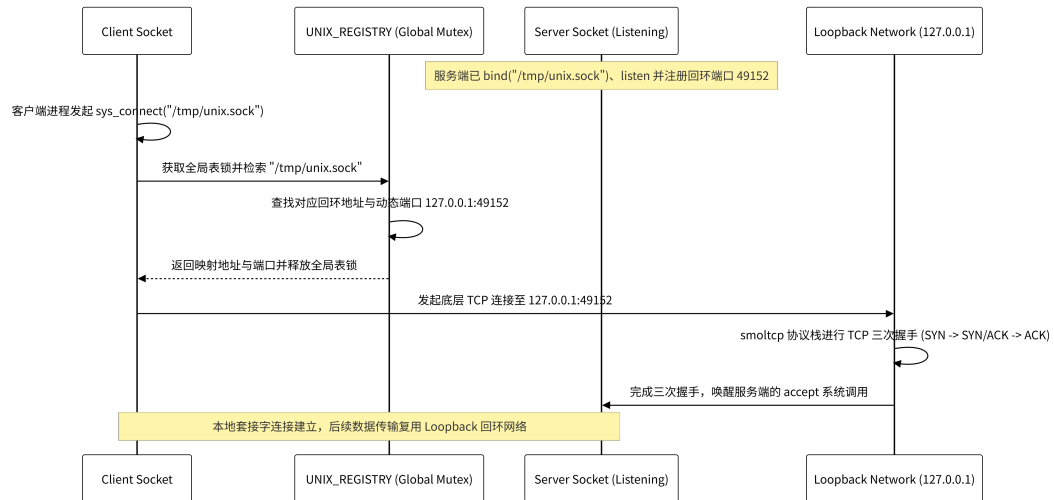
- `SIOCGIFCONF`：用于检索网络接口的配置列表。为了让 `ifconfig` 等工具能够顺利执行查询，内核实现了该命令的静态响应，向用户态传入的 `ifconf` 结构体中的 `ifreq` 数组格式化回填环回接口 `lo` 与以太网网卡 `eth0` 的静态配置，满足用户态的网络信息初始化与呈现需求。
- 其它接口控制命令，如 `SIOCGIFFLAGS`、`SIOCGIFHWADDR`、`SIOCGIFMTU`、`SIOCGIFTXQLEN`：内核同样提供了静态 Stub 支持，返回硬编码的网卡 Flags、MTU 值、MAC 地址和发送队列长度，从而实现了对于 `ifconfig` 工具完整查询链的静态兼容。

6.4 本地套接字退化与多态映射机制 AF_UNIX

PulseOS 在处理 UNIX 域套接字 `AF_UNIX` 时采用了极度简化的方案：针对普通命名套接字，退化为回环网络以减少内核协议栈的独立开发开销；针对成对匿名套接字，则使用专用的内存管道缓冲区。

- 普通套接字退化与映射：
 - 套接字映射：在 `sys_socket` 系统调用中，如果请求 `AF_UNIX` 域，则根据类型直接映射为普通的 `TcpSocket`（流式 `SOCK_STREAM`）或 `UdpSocket`（数据报 `SOCK_DGRAM`）。
 - 地址绑定 `bind`：如果是 `pathname` 类型的套接字，内核在 VFS 对应的路径下创建一个空文件节点，以便文件系统可见。随后将套接字绑定到本地回环网络 `127.0.0.1:0`，由底层 `smoltcp` 随机分配一个空闲的本地端口。最后，将套接字路径与本地端口的映射关系注册进全局的 `UNIX_REGISTRY` 中。
 - 建立连接 `connect`：当另一个套接字连接到该 UNIX 路径时，内核查询全局 `UNIX_REGISTRY`，检索对应的本地回环端口号，并在底层网络协议栈上直接发起对 `127.0.0.1:端口` 的 TCP 或 UDP 连接。

客户端连接本地 UNIX 域套接字服务端的过程时序如下：



- 与通过 `sys_socket` 创建的单体套接字不同，通过 `sys_socketpair` 系统调用创建的匿名对套接字，在内核中由 `SocketInner::Local` 分支接管。其底层直接构建了基于 `LocalSocketRingBuffer` 的物理环形缓冲区，读写双方在内存中直接拷贝数据，从而提供无网络栈参与的极速本地进程间通信支持。

第 7 章 阶段性成果与实现参考

7.1 阶段性成果

截止 2026.6.30, PulseOS 在 RISC-V 64 和 LoongArch 64 两个架构的 QEMU 模拟器上均完全通过了所有初赛测例, 并在部分性能测试中取得了较为领先的分数。

7.2 问题与经验

在 PulseOS 的开发过程中也遇到了大量的问题。

例如, 在开发初期, ArceOS 默认使用的 `lwext4_rust` 在线上测评环境中由于依赖 `musl` 工具链而无法编译。面对这种情况, 我们选择迁移至 `ext4_rs`, 并大幅修改了 `axfs/vfs`, 但也在该阶段引入了较多文件系统缺陷。在初赛末期, 我们又选择将 `ext4_rs` 迁移至 `ext4plus` 并对 `axfs/vfs` 进行了重构。PulseOS 在文件系统设计上经历了频繁的重构与迁移, 这使我们深刻认识到初期架构选型的重要性。

此外, 由于我们对 QEMU 以及硬件底层启动流程认知不足, 导致最初无法解决 LoongArch 64 在 QEMU10 上无法正常启动的问题, 最后借助 AI 进行了排查与解决。这也显示出硬件底层问题定位的困难性。

我们在 PulseOS 开发过程中积累的经验也是宝贵的, 从内核各个模块的设计实现, 到 AI 辅助下的开发与调试, 我们都借以积累了很多经验。

7.3 与同类系统亮点对比

我们将 PulseOS 的核心机制设计与同类系统进行量化与设计对比, 提炼出以下核心技术亮点:

7.3.1 物理页帧元数据管理

- 同类系统设计: 在 StarryOS 中, COW 的物理页面引用计数使用一个全局的 `BTreeMap<PhysAddr, Arc<SpinNoIrq<FrameRefCnt>>>` 来管理。这种设计会导致两类明显的开销: 一是全局 BTree 锁的高频争用, 导致并发 `clone / fork` 时锁延迟增加; 二是为每一个物理页动态分配一个 `Arc` 引用计数堆块, 造成了较大的内存分配开销, 查找复杂度为 $O(\log N)$ 。

- **PulseOS 亮点实现：** PulseOS 预先分配了扁平数组结构的物理页帧表 `FrameTable`。对物理地址的引用计数查询与修改可以通过 $O(1)$ 的计算直接定位，不需要遍历任何数据结构。同时，引用计数更新全部采用无锁原子操作，彻底消除了全局锁竞争。

7.3.2 用户态内存拷贝安全隔离

- **同类系统设计：** RocketOS 等系统在处理用户态内存拷贝时，在验证指针范围后直接在内核态通过用户虚拟地址进行解引用拷贝。如果用户虚拟地址所映射的物理页无效，解引用将直接导致内核态硬件崩溃；且依赖于内核-用户混合地址空间，隔离性较弱。
- **PulseOS 亮点实现：** PulseOS 实现了一套物理页转换直存拷贝机制：内核从不直接解引用用户虚拟地址，而是通过页表查询将用户地址翻译为物理地址，并通过安全内核映射 `phys_to_virt` 对物理内存页进行内核拷贝。同时，设计了内核态显式软件缺页懒加载机制，以两阶段读写锁升级机制手动进行页面填充，从根本上杜绝了内核态直接解引用用户虚地址带来的崩溃隐患与越界安全漏洞。

7.3.3 纯 Rust 文件系统与异步/中断 I/O 架构

- **同类系统设计：** StarryOS 等基于 ArceOS 的操作系统，其 `ext4` 文件系统封装了底层的 `lwext4` C 语言库。该方案在编译时涉及 C 外部依赖，跨语言的 FFI 调用带来了额外的性能损耗；另一方面，C 语言库无法与 Rust 内核的协程/异步调度器实现无缝协同。
- **PulseOS 亮点实现：** PulseOS 采用了纯 Rust 语言实现的 `ext4plus` 库，从根本上消除了外部 C 依赖带来的安全隐患与移植壁垒。同时，`ext4plus` 利用 `maybe-async` 机制保留了完整的异步 I/O 接口定义，为后续 PulseOS 的异步化与中断驱动改造创造了条件。

7.4 未来工作展望

在决赛阶段，我们将继续进行以下工作：

1. 完善终端功能。
2. 向异步化方向改进，进一步优化 PulseOS 的性能。
3. 适配多核环境。
4. 适配龙芯 2K1000 星云板以及 VisionFive2 星光二代板。

5. 完善网络模块。

7.5 实现参考

- RocketOS: 借鉴了其批量懒分配的多页映射机制。
- undefinedOS: 在前期实现共享内存时参考了其共享内存多映射设计。
- StarryOS: 复用了 `starry-vdso` 组件并加以改进, 该组件也是我在 2025 秋冬季开源操作系统训练营中完成的工作。也正因如此, PulseOS 在设计上有很多地方借鉴了 StarryOS。
- qperf: 使用了上一届比赛中 Starry Mix 开发的 QEMU 性能分析插件对 PulseOS 进行了性能热点分析。

第 8 章 AI 使用说明

8.1 工具

CodeX、Gemini CLI、Antigravity、Jules、GitHub Copilot、CodeRabbit

8.2 模型

GPT-5 系列模型、Gemini 3.1/3.5 系列模型、Claude 4.6 Sonnet 与 DeepSeek V4 系列模型

8.3 使用记录

为了保障项目的可追溯性与开发透明度，以及符合比赛要求，我们对 AI 辅助开发与调优的过程进行了分类归档，主要记录如下：

- 记录了 AI 在各个内核模块编写过程中的部分交互过程，归档于项目根目录下的 `records/ai-logs/` 目录中。
- 记录了使用 AI 辅助分析 `qperf` 火焰图并定位系统性能瓶颈的交互过程，归档于 `records/qperf-analyze/` 目录中。
- 在团队协作中，我们使用了 CodeX、Gemini Code Assist、GitHub Copilot 与 CodeRabbit 进行持续集成评审。具体的 Review 过程与反馈已完整保留在 [PulseOS Pull Requests](#) 中已合并 PR 的评审评论内。
- 本地套接字退化、Netlink 状态回填及底层协议栈对接等网络模块的大部分编码与重构工作是由 AI 完成的。

在 LTP 适配过程中，由于 AI 对话数量较多，不便于全部展示，仅给出我们使用的以下模式：

```
user: [ltp运行日志]请你根据以上运行日志，检查测例代码[测例代码]，制订修改计划进行修改以尽可能多的通过这些测试
ai: [修改方案]
user: [review 修改方案]
ai: [开始执行]
```